

Имитационное моделирование

**Лабораторная работа №7. Дискретно-событийное
моделирование: M/M/c и модель Росса**

Ведьмина Александра Сергеевна

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
3.1	Модель М/М/с	7
3.2	Модель Росса	8
3.3	Организация вычислительного проекта	8
4	Выполнение лабораторной работы	10
4.1	Подготовка проекта	10
4.2	Структура проекта	14
4.3	Исходный код проекта	14
4.4	Результаты для модели М/М/с	19
4.5	Результаты для модели Росса	25
4.6	Сформированные производные артефакты	32
5	Выводы	34
	Список литературы	35

Список иллюстраций

4.1	Создание проекта <code>lab_07_models</code> через DrWatson	10
4.2	Обновление окружения проекта	11
4.3	Добавление зависимостей CSV, DataFrames, IJulia, Literate, Plots	12
4.4	Инициализация проекта и успешный запуск <code>test/runtests.jl</code>	12
4.5	Запуск <code>scripts/generate.jl</code>	13
4.6	Запуск <code>scripts/discrete_event_clean.jl</code>	13
4.7	Запуск <code>scripts/discrete_event_param_clean.jl</code>	14
4.8	Временная динамика числа заявок в системе, очереди и обслуживании	21
4.9	Гистограмма времени ожидания в очереди	22
4.10	Загрузка двух серверов в базовом эксперименте	23
4.11	Сравнение симуляционных и аналитических метрик $M/M/2$	23
4.12	Параметрический прогон $M/M/c$ по числу серверов	24
4.13	Число исправных и работающих машин во времени	26
4.14	Загрузка ремонтников и очередь на ремонт	26
4.15	Гистограмма времени до падения системы	27
4.16	Влияние числа ремонтников на среднее время до отказа	28
4.17	Среднее время до отказа при разном числе рабочих машин	29
4.18	Загрузка ремонтной службы при разном числе рабочих машин	30
4.19	Расширенный прогон по числу ремонтников: время до отказа	31
4.20	Расширенный прогон по числу ремонтников: загрузка и очередь	32

Список таблиц

4.1	Сравнение базовой симуляции М/М/2 с формулами Эрланга С . . .	20
4.2	Влияние числа каналов на вероятность ожидания и среднюю задержку	24
4.3	Базовый эксперимент для модели Росса	25
4.4	Влияние числа ремонтников на надёжность системы	28
4.5	Влияние числа рабочих машин на надёжность системы	30

1 Цель работы

Изучить дискретно-событийное моделирование на примере двух систем: многоканальной очереди М/М/с и модели Росса для ремонтируемой системы с резервом, реализовать их в структуре DrWatson, выполнить базовые и параметрические прогоны, подготовить автоматические тесты, сгенерировать clean-, qmd- и ipynb-версии сценариев, а также оформить отчёт и презентацию по реальным вычислительным артефактам проекта.

2 Задание

В ходе лабораторной работы требовалось:

1. создать рабочий каталог `lab_07_models` в структуре `DrWatson`;
2. установить зависимости `CSV`, `DataFrames`, `DrWatson`, `IJulia`, `Literate`, `Plots`;
3. реализовать модель $M/M/c$ со сравнением симуляции и аналитических формул Эрланга C ;
4. реализовать модель Росса с несколькими ремонтниками;
5. выполнить прогон для разного числа серверов, машин и ремонтников;
6. провести мониторинг загрузки обслуживающих устройств и длины очереди;
7. построить графики временной динамики и параметрических серий;
8. подготовить `literate`-скрипты и сгенерировать из них `clean`-, `qmd`- и `ipynb`-представления;
9. вставить исходный код, графики и скриншоты выполнения в отчёт и презентацию.

3 Теоретическое введение

3.1 Модель М/М/с

По классификации Кендалла система М/М/с описывает очередь с пуассоновским входящим потоком, экспоненциальным временем обслуживания и с одинаковыми обслуживающими каналами [2; 3]. Основные параметры:

- λ — интенсивность входящего потока заявок;
- μ — интенсивность обслуживания одним каналом;
- c — число параллельных серверов;
- $\rho = \lambda / (c \cdot \mu)$ — нагрузка на один канал.

Для стационарного режима необходимо условие $\rho < 1$. В этом случае вероятность пустой системы и вероятность ожидания находятся по формуле Эрланга С:

$$P_0 = \left[\sum_{n=0}^{c-1} \frac{(\lambda/\mu)^n}{n!} + \frac{(\lambda/\mu)^c}{c!(1-\rho)} \right]^{-1},$$

$$P_{\text{wait}} = \frac{(\lambda/\mu)^c}{c!(1-\rho)} P_0.$$

После этого через формулы Литтла определяются среднее число заявок в очереди, среднее ожидание и среднее время пребывания в системе:

$$L_q = P_{\text{wait}} \frac{\rho}{1-\rho}, \quad W_q = \frac{L_q}{\lambda}, \quad W = W_q + \frac{1}{\mu}, \quad L = \lambda W.$$

3.2 Модель Росса

Модель Росса рассматривает систему из N основных машин, S резервных машин и ремонтной службы [5]. Пока резерв существует, отказавшая рабочая машина мгновенно заменяется исправной запасной, а сломанная уходит в ремонт. Если резерв исчерпан, следующий отказ приводит к падению системы.

В реализованном варианте состояние задаётся числом исправных машин. При этом:

- число работающих машин равно $\min(N, i)$;
- число запасных равно $\max(0, i - N)$;
- число машин в ремонте равно $N + S - i$.

Аналитическое среднее время до отказа вычисляется через систему линейных уравнений для непрерывного марковского процесса. В дискретно-событийной симуляции моделируются два типа событий:

- отказ рабочей машины;
- завершение ремонта.

Дополнительно контролируются:

- загрузка ремонтников;
- средняя длина очереди на ремонт;
- число исправных машин во времени;
- среднее время до падения системы.

3.3 Организация вычислительного проекта

В работе использовались:

- DrWatson для воспроизводимой структуры научного проекта [1];

- `Literate` и подход `literate programming` для получения из одного источника `clean`-, `qmd`- и `ipynb`-версий сценариев [4];
- `CSV` и `DataFrames` для сохранения результатов;
- `Plots` для построения графиков;
- `Quarto` для интеграции вычислительных материалов в отчёт.

4 Выполнение лабораторной работы

4.1 Подготовка проекта

4.1.1 Создание проекта через DrWatson

Сначала в каталоге `labs/lab07` был создан рабочий каталог `lab_07_models` с помощью функции `initialize_project(...)` из пакета `DrWatson`.



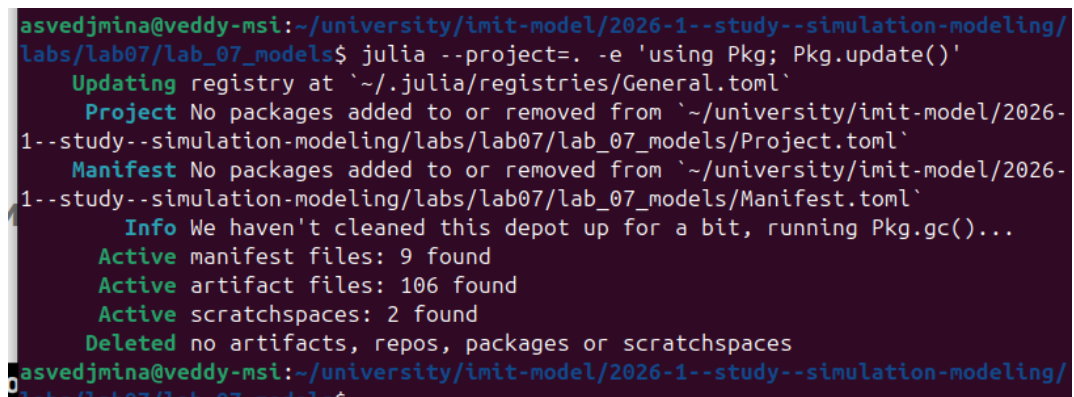
```
asvedjmina@veddy-msi:~/university/imit-model/2026-1--study--simulation-modeling/
labs/lab07$ julia -e 'using DrWatson; initialize_project("lab_07_models", author
s="Ведьмина Александра Сергеевна")'
  Activating new project at `~/university/imit-model/2026-1--study--simulation-m
odeling/labs/lab07/lab_07_models`
  Resolving package versions...
  Updating `~/university/imit-model/2026-1--study--simulation-modeling/labs/la
b07/lab_07_models/Project.toml`
    [634d3b9d] + DrWatson v2.19.1
  Updating `~/university/imit-model/2026-1--study--simulation-modeling/labs/la
b07/lab_07_models/Manifest.toml`
    [0b6fb165] + ChunkCodecCore v1.0.1
    [4c0bbe4] + ChunkCodecLibZlib v1.0.0
    [55437552] + ChunkCodecLibZstd v1.0.0
    [634d3b9d] + DrWatson v2.19.1
    [5789e2e9] + FileIO v1.19.0 [loaded: v1.18.0]
    [076d061b] + HashArrayMappedTries v0.2.0
    [033835bb] + JLD2 v0.6.4 [loaded: v0.6.3]
    [692b3bcd] + JLLWrappers v1.8.0 [loaded: v1.7.1]
    [1914dd2f] + MacroTools v0.5.16
    [bac558e1] + OrderedCollections v1.8.1
    [aea7be01] + PrecompileTools v1.3.4 [loaded: v1.3.3]
    [21216c6a] + Preferences v1.5.2 [loaded: v1.5.1]
```

Рисунок 4.1: Создание проекта `lab_07_models` через `DrWatson`

На этом шаге были автоматически созданы файлы `Project.toml`, `Manifest.toml` и стандартная структура проекта.

4.1.2 Обновление окружения и установка зависимостей

После создания проекта было обновлено окружение Julia и добавлены зависимости, необходимые для моделирования и генерации производных форматов.



```
asvedjmina@veddy-msi:~/university/imit-model/2026-1--study--simulation-modeling/
labs/lab07/lab_07_models$ julia --project=. -e 'using Pkg; Pkg.update()'
Updating registry at `~/.julia/registries/General.toml`
Project No packages added to or removed from `~/university/imit-model/2026-
1--study--simulation-modeling/labs/lab07/lab_07_models/Project.toml`
Manifest No packages added to or removed from `~/university/imit-model/2026-
1--study--simulation-modeling/labs/lab07/lab_07_models/Manifest.toml`
Info We haven't cleaned this depot up for a bit, running Pkg.gc()...
Active manifest files: 9 found
Active artifact files: 106 found
Active scratchspaces: 2 found
Deleted no artifacts, repos, packages or scratchspaces
asvedjmina@veddy-msi:~/university/imit-model/2026-1--study--simulation-modeling/
labs/lab07/lab_07_models$
```

Рисунок 4.2: Обновление окружения проекта

```

asvedjmina@veddy-msi:~/university/imit-model/2026-1--study--simulation-modeling/
labs/lab07/lab_07_models$ julia --project=. -e 'using Pkg; Pkg.add(["CSV", "Data
Frames", "DrWatson",
"IJulia", "Literate", "Plots"])'
    Resolving package versions...
    Updating `~/university/imit-model/2026-1--study--simulation-modeling/labs/la
b07/lab_07_models/Project.toml`
 [336ed68f] + CSV v0.10.16
 [a93c6f00] + DataFrames v1.8.2
 [7073ff75] + IJulia v1.34.4
 [98b081ad] + Literate v2.21.0
 [91a5bcdd] + Plots v1.41.6
    Updating `~/university/imit-model/2026-1--study--simulation-modeling/labs/la
b07/lab_07_models/Manifest.toml`
 [66dad0bd] + AliasTables v1.1.3
 [d1d4a3ce] + BitFlags v0.1.9
 [336ed68f] + CSV v0.10.16
 [944b1d66] + CodecZlib v0.7.8
 [35d6a980] + ColorSchemes v3.31.0
 [3da002f7] + ColorTypes v0.12.1
 [c3611d14] + ColorVectorSpace v0.11.0
 [5ae59095] + Colors v0.13.1

```

Рисунок 4.3: Добавление зависимостей CSV, DataFrames, IJulia, Literate, Plots

В результате проект получил полный воспроизводимый набор библиотек, включая инструменты для анализа таблиц, визуализации, literate-генерации и работы с ноутбуками.

4.1.3 Инициализация, предкомпиляция и тесты

После установки зависимостей были выполнены `Pkg.instantiate()`, `Pkg.precompile()` и автоматические тесты проекта.

```

asvedjmina@veddy-msi:~/university/imit-model/2026-1--study--simulation-modeling/
labs/lab07/lab_07_models$ julia --project=. -e 'using Pkg; Pkg.instantiate()'
asvedjmina@veddy-msi:~/university/imit-model/2026-1--study--simulation-modeling/
labs/lab07/lab_07_models$ julia --project=. -e 'using Pkg; Pkg.precompile()'
asvedjmina@veddy-msi:~/university/imit-model/2026-1--study--simulation-modeling/
labs/lab07/lab_07_models$ julia --project=. test/runtests.jl
Test Summary:      | Pass Total Time
lab_07_models tests |  27    27  2.7s
asvedjmina@veddy-msi:~/university/imit-model/2026-1--study--simulation-modeling/
labs/lab07/lab_07_models$

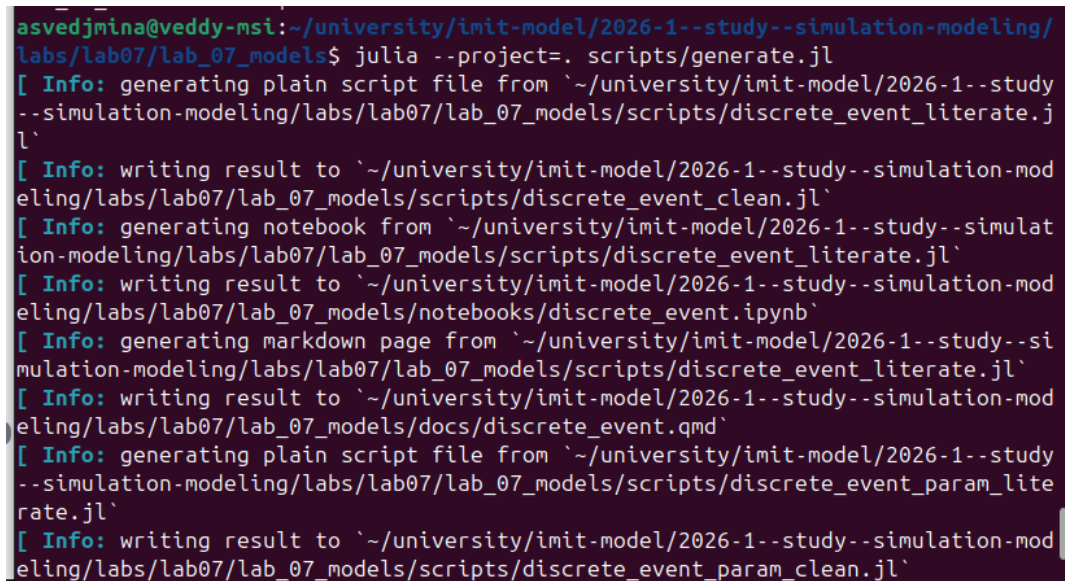
```

Рисунок 4.4: Инициализация проекта и успешный запуск `test/runtests.jl`

Набор тестов включает 27 проверок. Он подтверждает корректность аналитики М/М/с, симуляции очереди, аналитического среднего времени до отказа в модели Росса, а также параметрических серий.

4.1.4 Генерация производных форматов

Затем был выполнен генератор `scripts/generate.jl`, который создал clean-версии скриптов, qmd-документы и `ipynb`-ноутбуки.



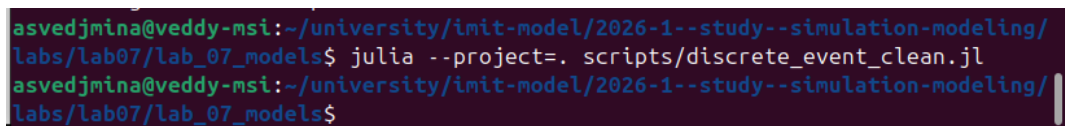
```
asvedjmina@veddy-msi:~/university/imit-model/2026-1--study--simulation-modeling/
labs/lab07/lab_07_models$ julia --project=. scripts/generate.jl
[ Info: generating plain script file from `~/university/imit-model/2026-1--study
--simulation-modeling/labs/lab07/lab_07_models/scripts/discrete_event_literate.j
l`
[ Info: writing result to `~/university/imit-model/2026-1--study--simulation-mod
eling/labs/lab07/lab_07_models/scripts/discrete_event_clean.jl`
[ Info: generating notebook from `~/university/imit-model/2026-1--study--simulat
ion-modeling/labs/lab07/lab_07_models/scripts/discrete_event_literate.jl`
[ Info: writing result to `~/university/imit-model/2026-1--study--simulation-mod
eling/labs/lab07/lab_07_models/notebooks/discrete_event.ipynb`
[ Info: generating markdown page from `~/university/imit-model/2026-1--study--si
mulation-modeling/labs/lab07/lab_07_models/scripts/discrete_event_literate.jl`
[ Info: writing result to `~/university/imit-model/2026-1--study--simulation-mod
eling/labs/lab07/lab_07_models/docs/discrete_event.qmd`
[ Info: generating plain script file from `~/university/imit-model/2026-1--study
--simulation-modeling/labs/lab07/lab_07_models/scripts/discrete_event_param_lite
rate.jl`
[ Info: writing result to `~/university/imit-model/2026-1--study--simulation-mod
eling/labs/lab07/lab_07_models/scripts/discrete_event_param_clean.jl`
```

Рисунок 4.5: Запуск `scripts/generate.jl`

Тем самым проект стал содержать единый модуль модели, два `literate`-источника и три производных формата для каждого сценария.

4.1.5 Выполнение вычислительных сценариев

После генерации были запущены два `clean`-сценария: базовый и параметрический.



```
asvedjmina@veddy-msi:~/university/imit-model/2026-1--study--simulation-modeling/
labs/lab07/lab_07_models$ julia --project=. scripts/discrete_event_clean.jl
asvedjmina@veddy-msi:~/university/imit-model/2026-1--study--simulation-modeling/
labs/lab07/lab_07_models$
```

Рисунок 4.6: Запуск `scripts/discrete_event_clean.jl`

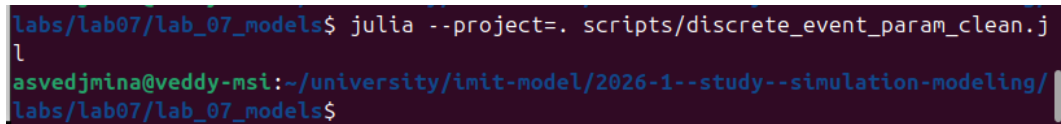
A terminal window with a dark background. The prompt is 'labs/lab07/lab_07_models\$'. The command entered is 'julia --project=. scripts/discrete_event_param_clean.jl'. The output shows the user's name and email 'asvedjmina@veddy-msi:~/university/imit-model/2026-1--study--simulation-modeling/' followed by the prompt 'labs/lab07/lab_07_models\$'.

Рисунок 4.7: Запуск `scripts/discrete_event_param_clean.jl`

Именно эти запуски сформировали графики в каталоге `plots/` и таблицы в каталоге `data/`, которые затем были использованы в настоящем отчёте.

4.2 Структура проекта

По итогам выполнения лабораторной работы были получены следующие основные компоненты проекта:

- `src/DiscreteEventModels.jl` — общий модуль моделей и графиков;
- `scripts/discrete_event_literate.jl` — базовый сценарий;
- `scripts/discrete_event_param_literate.jl` — параметрический сценарий;
- `scripts/generate.jl` — генерация `clean`-, `qmd`- и `ipynb`-форматов;
- `test/runtests.jl` — автоматические тесты;
- `data/*.csv` — результаты экспериментов;
- `plots/*.png` — все итоговые графики;
- `docs/*.qmd` и `notebooks/*.ipynb` — производные документы.

Такая организация полностью соответствует научно-ориентированному стилю работы с вычислительным проектом в DrWatson.

4.3 Исходный код проекта

4.3.1 Параметры моделей

В модуле используются две явные структуры параметров, разделяющие настройки очереди и модели надёжности:

```

Base.@kwdef struct MMcParameters
    lambda::Float64 = 0.9
    mu::Float64 = 0.5
    servers::Int = 2
    num_customers::Int = 500
end

Base.@kwdef struct RossParameters
    machines::Int = 10
    spares::Int = 3
    repairers::Int = 1
    failure_mean::Float64 = 100.0
    repair_mean::Float64 = 1.0
end

```

Благодаря этому обе модели удобно запускать как в базовом, так и в параметрическом режимах.

4.3.2 Аналитика для М/М/с

Ключевой фрагмент аналитического расчёта реализует формулу Эрланга С и все производные характеристики системы:

```

rho = lambda / (servers * mu)
a = lambda / mu
...
tail = tail_base / (1.0 - rho)
p_wait = tail * p0
Lq = p_wait * rho / (1.0 - rho)
Wq = Lq / lambda

```

$$W = W_q + 1.0 / \mu$$

Именно эта функция используется как эталон при сравнении с результатами событийной симуляции.

4.3.3 Симуляция М/М/с

Событийный симулятор многоканальной очереди последовательно генерирует поступление заявок, выбирает ближайший свободный сервер и записывает индивидуальные характеристики каждой заявки:

```
ia_mean = 1.0 / params.lambda
svc_mean = 1.0 / params.mu

for customer_id in 1:params.num_customers
    arrival_time += sample_exponential(rng, ia_mean)
    service_time = sample_exponential(rng, svc_mean)
    server_id = argmin(server_available)
    free_at = server_available[server_id]
    service_start = max(arrival_time, free_at)
    departure_time = service_start + service_time
    waiting_time = service_start - arrival_time
    ...
end
```

Такая запись делает возможным дальнейший анализ гистограммы ожидания, загрузки серверов и временных ступенчатых графиков.

4.3.4 Аналитика модели Росса

Среднее время до отказа системы вычисляется из линейной системы для состояний, соответствующих числу исправных машин:

```
for row in 1:n_states
    healthy = params.machines + row - 1
    failures = min(
        params.machines, healthy
    ) / params.failure_mean
    repairs = min(
        params.repairers, broken
    ) / params.repair_mean
    A[row, row] = failures + repairs
end
```

Этот расчёт даёт ориентир, с которым затем сопоставляется усреднение по серии монте-карловских прогонов.

4.3.5 Симуляция модели Росса

Дискретно-событийная логика модели Росса реализуется как соревнование отказов и завершений ремонта:

```
while true
    failure_rate = params.machines / params.failure_mean
    gap_mean = 1.0 / failure_rate
    next_failure_time = time + sample_exponential(rng, gap_mean)
    next_repair_time = isempty(repair_completion_times) ?
        Inf : minimum(repair_completion_times)
    ...
end
```

```

    if next_failure_time <= next_repair_time
        if spares == 0
            break
        end
        ...
    else
        ...
    end
end

```

Благодаря этому можно отслеживать и сам момент падения системы, и динамику нагрузки на ремонтную службу.

4.3.6 Код `literate`-сценариев

Базовый `literate`-сценарий запускает основной эксперимент для двух моделей:

```

mmc_params = MMcParameters(
    lambda = 0.9,
    mu = 0.5,
    servers = 2,
)
base_rng = MersenneTwister(123)
mmc_df = simulate_mmc(mmc_params; rng = base_rng)

ross_params = RossParameters(
    machines = 10,
    spares = 3,
    repairers = 2,
)

```

```
ross_result = simulate_ross(ross_params; seed = 150)
```

Параметрический сценарий выполняет серии запусков:

```
mmc_sweep = mmc_server_sweep(  
  1:5;  
  lambda = 0.45,  
  mu = 0.5,  
  seed = 900,  
)  
ross_machine_df = ross_machine_sweep(  
  6:2:14;  
  spares = 3,  
  repairers = 2,  
)  
ross_repairer_df = ross_repairer_sweep(  
  1:4;  
  machines = 10,  
  spares = 3,  
)
```

Тем самым исходный код сценариев явно привязан к тем таблицам и графикам, которые приведены ниже.

4.4 Результаты для модели М/М/с

4.4.1 Основные численные показатели

В базовом эксперименте использовались параметры:

- $\rho = 0.9$;

- $\rho = 0.5$;
- $c = 2$;
- число заявок 800.

Для этой конфигурации загрузка одного канала составила $\rho = 0.9$, то есть система работала в сильно нагруженном режиме. Сравнение однократной симуляции с аналитическими формулами приведено в табл. 4.1.

Таблица 4.1: Сравнение базовой симуляции M/M/2 с формулами Эрланга С

Метрика	Симуляция	Аналитика
$P(\text{wait})$	0.8888	0.8526
W_q	16.7378	8.5263
W	18.8077	10.5263
L_q	15.0640	7.6737
L	16.9269	9.4737

Однократная длинная траектория качественно подтверждает аналитические выводы, но при высокой загрузке даёт заметно более тяжёлую реализацию очереди. Это нормально для конечного стохастического прогона без дополнительного усреднения.

4.4.2 Временная динамика очереди и распределение ожидания

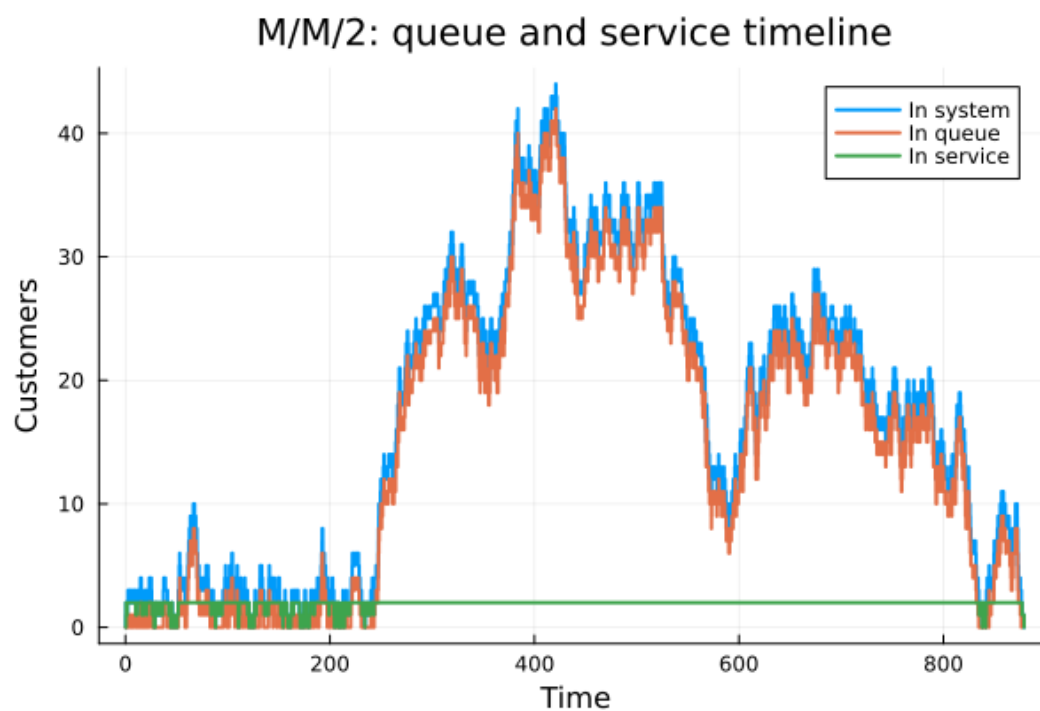


Рисунок 4.8: Временная динамика числа заявок в системе, очереди и обслуживании

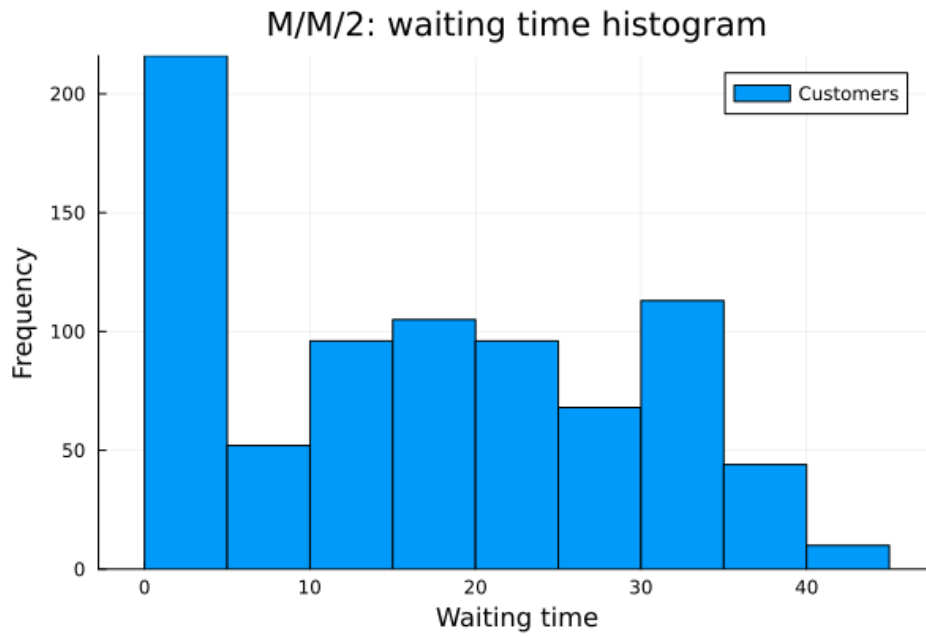


Рисунок 4.9: Гистограмма времени ожидания в очереди

Из таблицы `mms_customers.csv` видно, что среднее время ожидания составило 16.74, а максимальное ожидание в реализованной траектории достигло 42.38 единиц времени. Поэтому гистограмма имеет выраженный правый хвост, характерный для перегруженной очереди.

4.4.3 Загрузка серверов и сопоставление с аналитикой

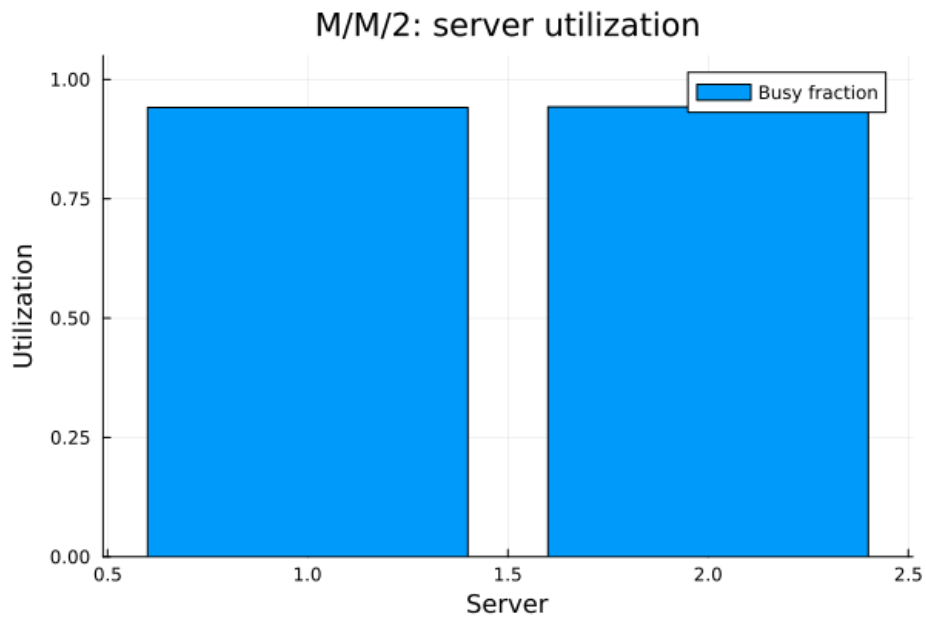


Рисунок 4.10: Загрузка двух серверов в базовом эксперименте

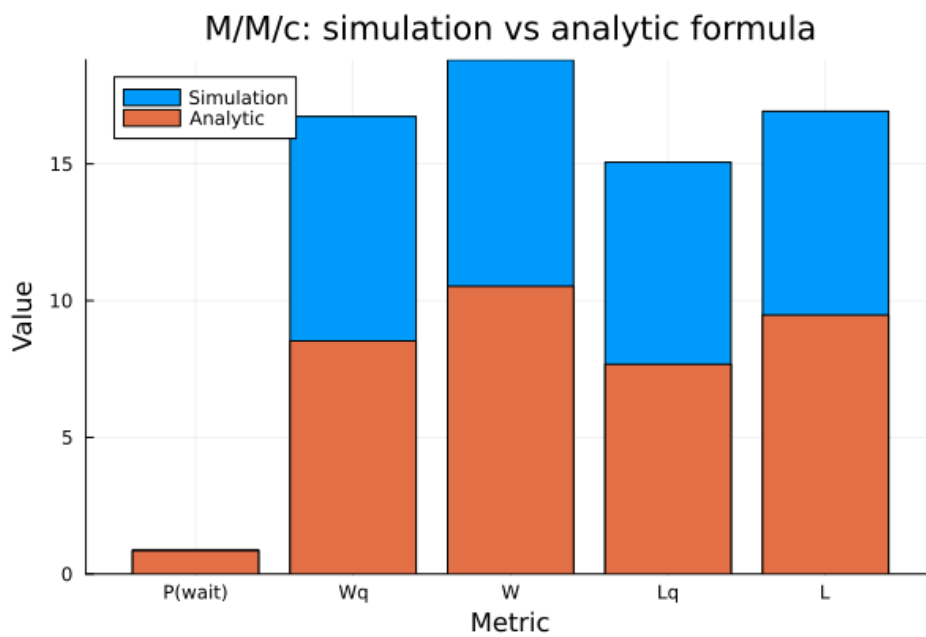


Рисунок 4.11: Сравнение симуляционных и аналитических метрик M/M/2

Средняя загрузка серверов в однократной реализации оказалась около 0.942, что хорошо согласуется с тем, что оба канала большую часть времени заняты.

4.4.4 Параметрический прогон по числу каналов

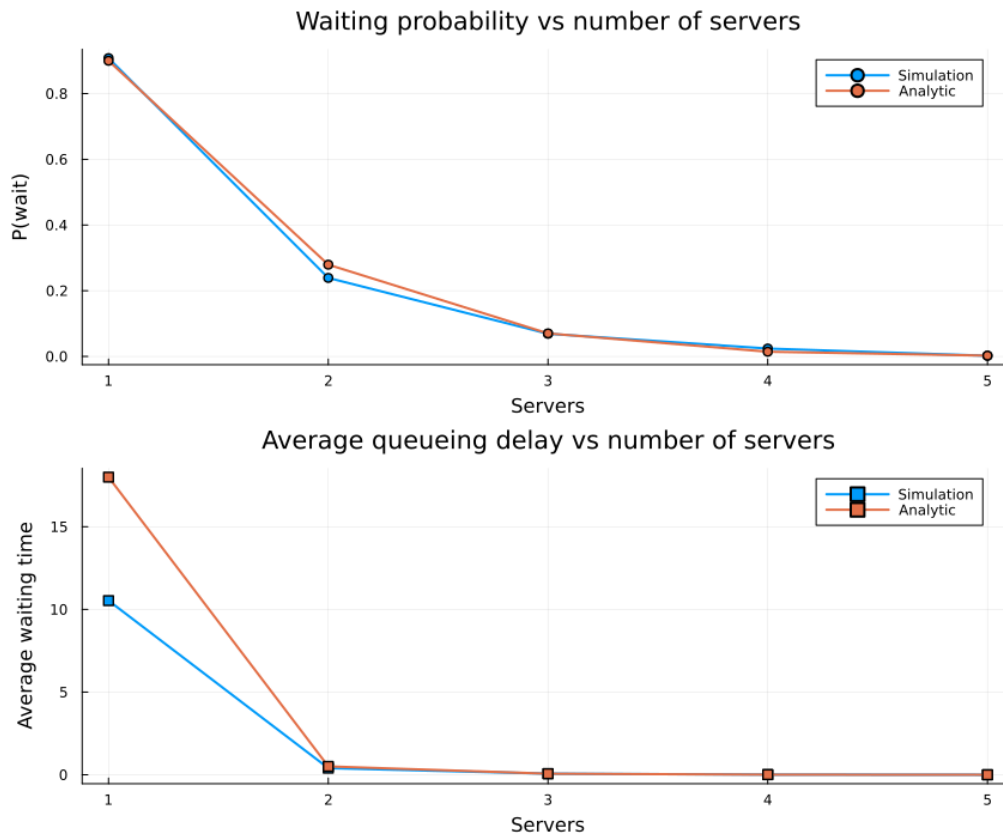


Рисунок 4.12: Параметрический прогон М/М/с по числу серверов

Таблица 4.2: Влияние числа каналов на вероятность ожидания и среднюю задержку

c	P(wait) сим.	P(wait) анал.	W_q сим.	W_q анал.
1	0.9080	0.9000	10.5326	18.0000
2	0.2390	0.2793	0.4012	0.5078
3	0.0690	0.0700	0.0697	0.0667

c	P(wait) сим.	P(wait) анал.	W_q сим.	W_q анал.
4	0.0240	0.0143	0.0113	0.0092
5	0.0020	0.0024	0.0010	0.0012

С увеличением числа серверов вероятность ожидания и среднее ожидание резко уменьшаются. Уже при $c = 3$ очередь становится кратковременной, а при $c = 5$ практически исчезает.

4.5 Результаты для модели Росса

4.5.1 Базовая конфигурация и сравнение с аналитикой

В базовом эксперименте использовались параметры:

- $N = 10$ рабочих машин;
- $S = 3$ резервные машины;
- 2 ремонтника;
- среднее время до отказа одной машины 100 часов;
- среднее время ремонта 1 час.

Среднее время до отказа по 40 стохастическим прогонам составило 47104.03 часа при аналитическом значении 46540.00 часа (табл. 4.3).

Таблица 4.3: Базовый эксперимент для модели Росса

N	S	ремонтники	аналитика	сим.	сим.	средняя
				среднее	СКО	
10	3	2	46540.00	47104.03	39113.41	0.0499
						0.0002

Низкая средняя длина очереди на ремонт показывает, что при двух ремонтниках ремонтная служба почти всегда успевает компенсировать отказы до исчерпания резерва.

4.5.2 Динамика исправных машин и нагрузки на ремонт

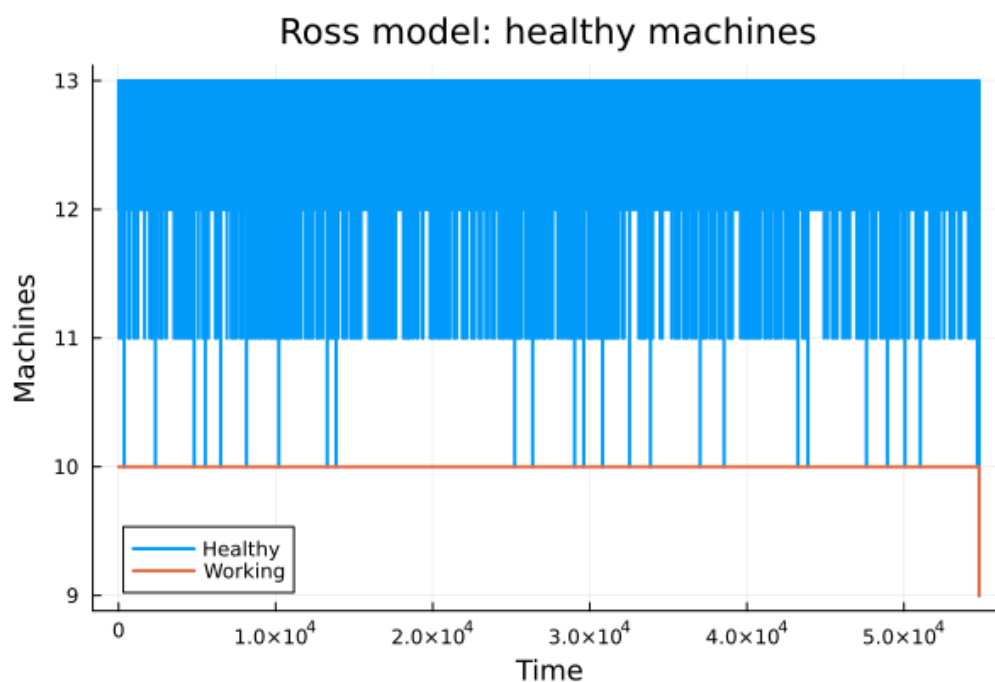


Рисунок 4.13: Число исправных и работающих машин во времени

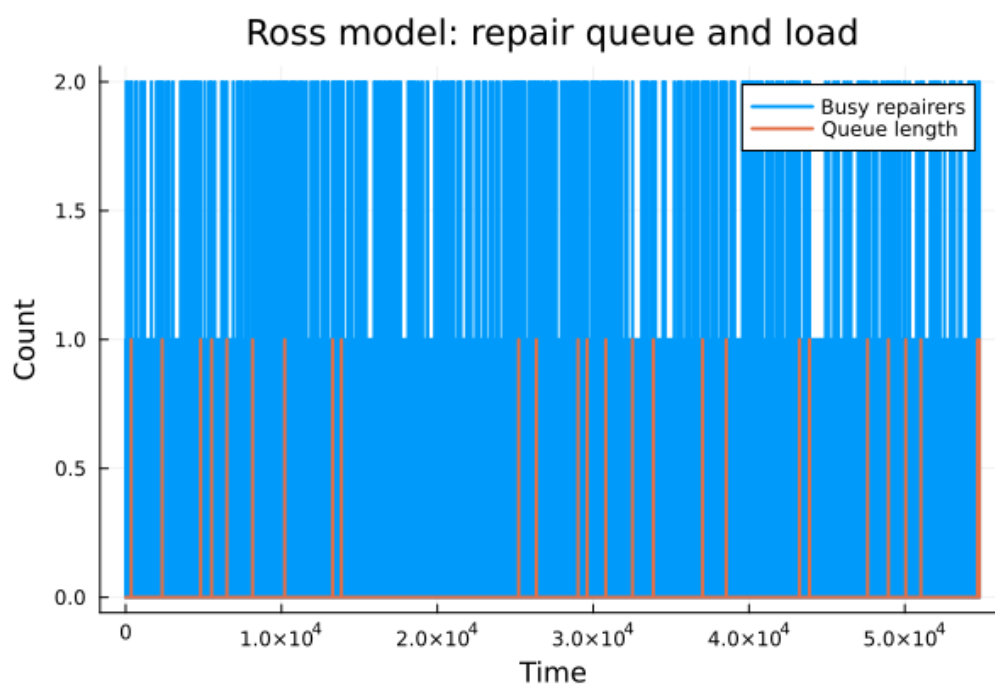


Рисунок 4.14: Загрузка ремонтников и очередь на ремонт

На первом графике видно, что запас исправных машин постепенно сокращается и восстанавливается после ремонтов. Второй график показывает, что очередь на ремонт возникает эпизодически и остаётся короткой.

4.5.3 Распределение времени до падения и влияние числа ремонтников

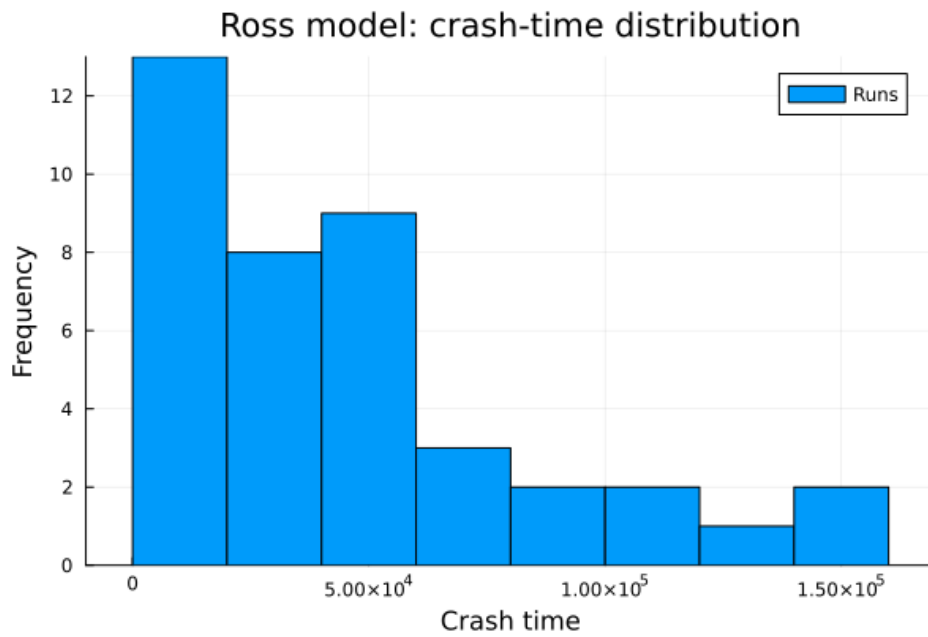


Рисунок 4.15: Гистограмма времени до падения системы

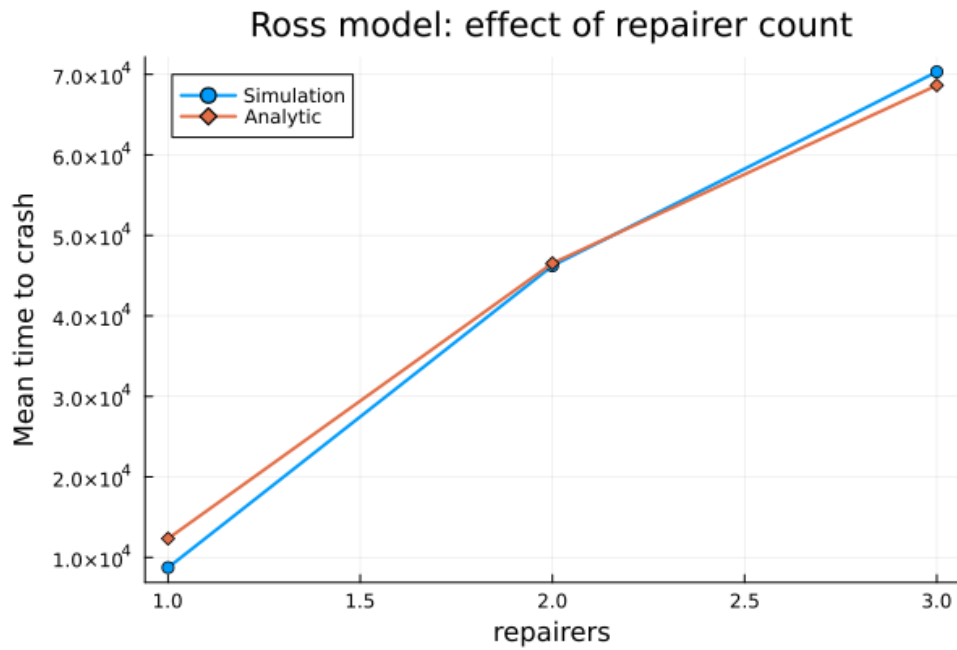


Рисунок 4.16: Влияние числа ремонтников на среднее время до отказа

Таблица 4.4: Влияние числа ремонтников на надёжность системы

Ремонтники	сим.				средняя
	среднее	сим. СКО	аналитика	загрузка	очередь
1	8731.21	8115.45	12340.00	0.1008	0.0121
2	46218.56	34553.73	46540.00	0.0500	0.0002
3	70321.34	68914.65	68640.00	0.0334	0.0000

Добавление второго ремонтника увеличивает среднее время до отказа почти на порядок, а третий ремонтник делает очередь практически нулевой.

4.5.4 Влияние числа машин

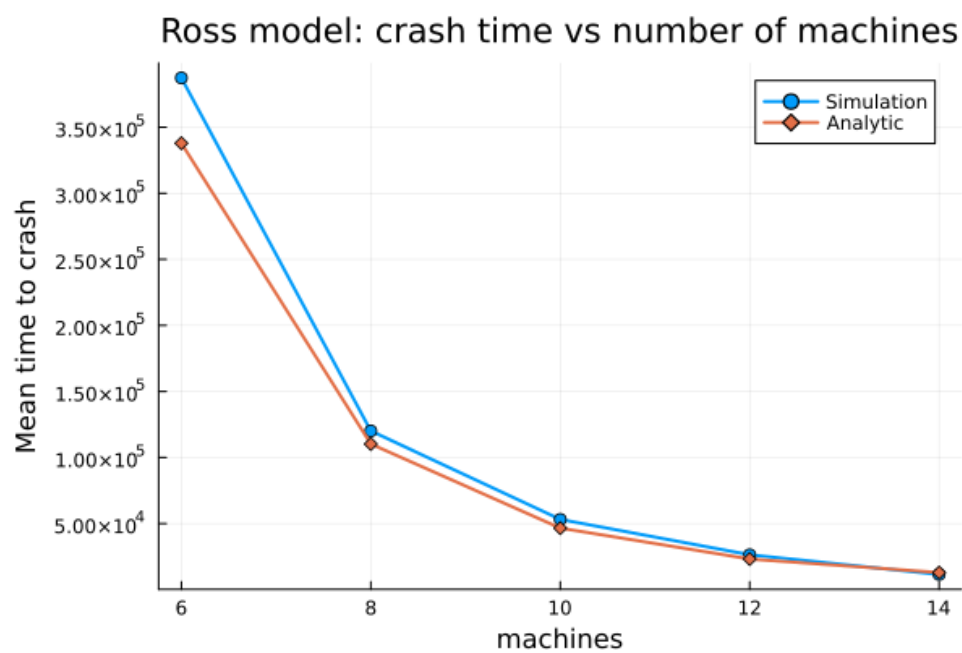


Рисунок 4.17: Среднее время до отказа при разном числе рабочих машин

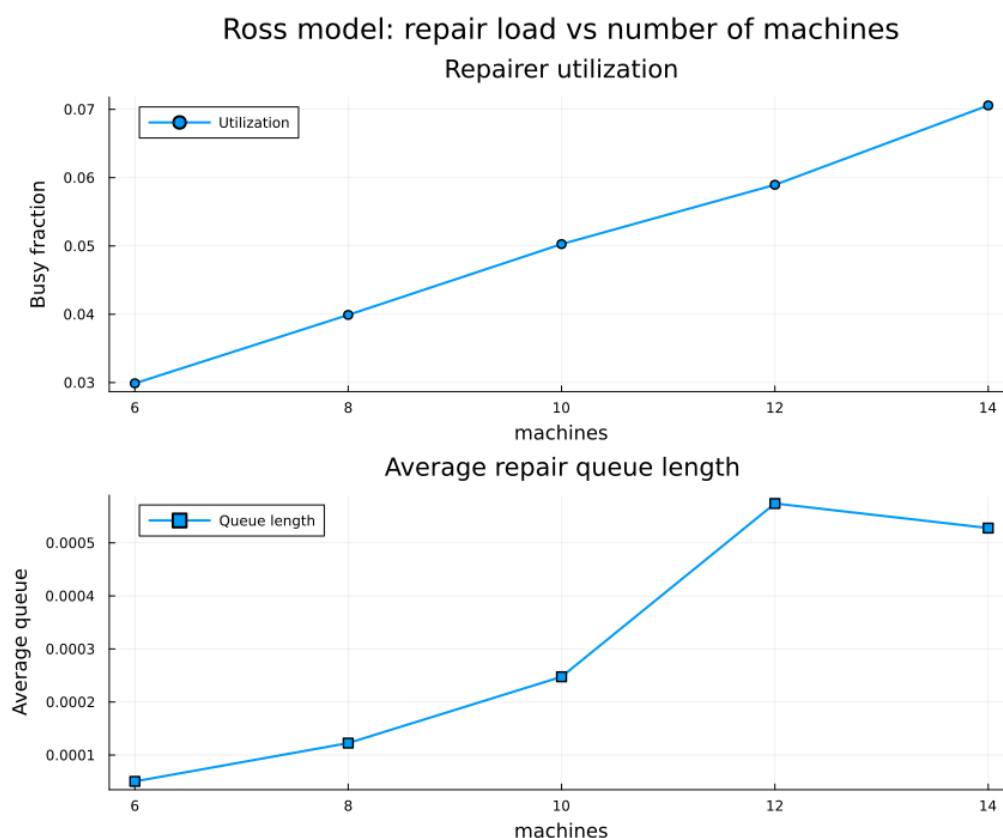


Рисунок 4.18: Загрузка ремонтной службы при разном числе рабочих машин

Таблица 4.5: Влияние числа рабочих машин на надёжность системы

Машины	сим. среднее	аналитика	загрузка	средняя очередь
6	387309.77	337875.31	0.0299	0.0001
8	120104.25	110206.25	0.0399	0.0001
10	53134.84	46540.00	0.0503	0.0002
12	26362.19	23142.90	0.0590	0.0006
14	11453.25	12882.59	0.0706	0.0005

Чем больше одновременно работающих машин при фиксированном резерве, тем выше интенсивность отказов системы в целом и тем быстрее резерв оказывается исчерпан.

4.5.5 Расширенный прогон по числу ремонтников

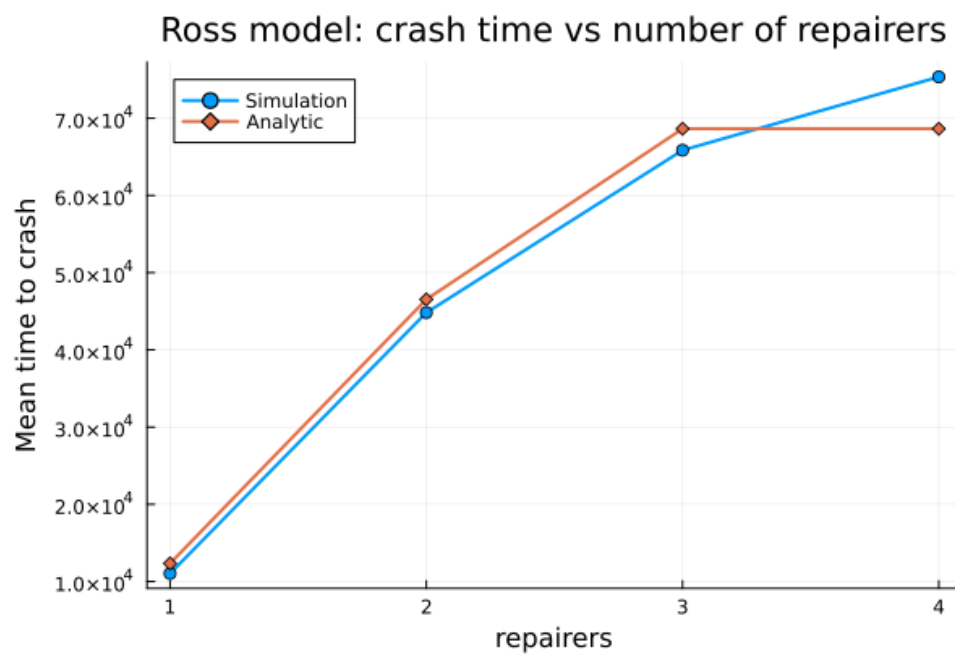


Рисунок 4.19: Расширенный прогон по числу ремонтников: время до отказа

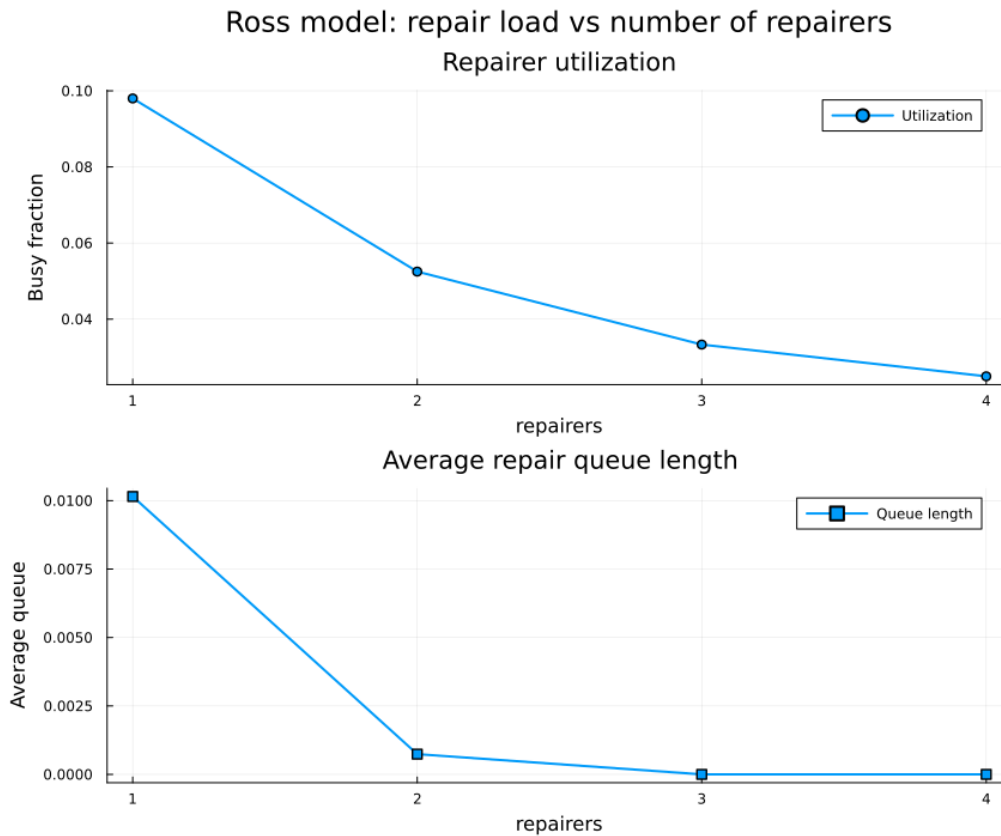


Рисунок 4.20: Расширенный прогон по числу ремонтников: загрузка и очередь

Расширенная серия по 1 : 4 ремонтникам подтверждает ту же закономерность: рост числа ремонтников увеличивает среднее время до отказа и одновременно снижает загрузку каждого ремонтника.

4.6 Сформированные производные артефакты

После выполнения лабораторной работы были автоматически сформированы:

- `scripts/discrete_event_clean.jl`;
- `scripts/discrete_event_param_clean.jl`;
- `docs/discrete_event.qmd`;
- `docs/discrete_event_param.qmd`;
- `notebooks/discrete_event.ipynb`;

- `notebooks/discrete_event_param.ipynb`.

Таким образом исходный `literate`-код действительно стал единым источником модели, вычислительных сценариев, документации и `notebook`-представления.

5 Выводы

В работе были реализованы две дискретно-событийные модели в общем модуле `DiscreteEventModels`: многоканальная очередь $M/M/c$ и модель Росса для ремонтируемой системы с резервом. Проект оформлен в структуре `DrWatson`, покрыт тестами и дополнен `literate`-сценариями с автоматической генерацией `clean`-, `qmd`- и `ipynb`-форматов.

Для $M/M/c$ было показано, что увеличение числа серверов резко уменьшает вероятность ожидания и среднее время в очереди. Для модели Росса было установлено, что число ремонтников и объём резерва критически влияют на среднее время до отказа: дополнительный ремонтник даёт сильный рост надёжности, а увеличение числа рабочих машин при фиксированном резерве, наоборот, ускоряет падение системы.

Все требуемые артефакты — исходный код, графики, CSV-таблицы, Quarto-документы и ноутбуки — были получены и интегрированы в отчёт.

Список литературы

1. DrWatson: the perfect sidekick to your scientific inquiries / G. Datseris [и др.] // Journal of Open Source Software. — 2020. — Т. 5, № 54. — С. 2673. — DOI: 10.21105/joss.02673.
2. Fundamentals of Queueing Theory / D. Gross [и др.]. — 4-е изд. — Wiley, 2008.
3. *Kendall D. G.* Stochastic Processes Occurring in the Theory of Queues and Their Analysis by the Method of the Imbedded Markov Chain // The Annals of Mathematical Statistics. — 1953. — Т. 24, № 3. — С. 338—354.
4. *Knuth D. E.* Literate Programming // The Computer Journal. — 1984. — Т. 27, № 2. — С. 97—111. — DOI: 10.1093/comjnl/27.2.97.
5. *Ross S. M.* Introduction to Probability Models. — 11-е изд. — Academic Press, 2014.